Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №5
з дисципліни «Сучасні мобільні операційні системи»

Тема: «Робота з даними – зовнішні файли»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Нестерук Андрій Олександрович

Дата здачі: 22.03.2026

КИЇВ – 2026

Мета роботи. Вивчити інструменти зберігання даних, а також роботу зі зовнішніми файлами.

Завдання 1. Вивчіть приклад підключення до мережі.

```
public void myClickHandler(View view) {
    ...

    ConnectivityManager connMgr = (ConnectivityManager) getSystemService
    ↪  (Context.CONNECTIVITY_SERVICE);
    NetworkInfo networkInfo = connMgr.getActiveNetworkInfo();

    if (networkInfo! = null && networkInfo.isConnected()) {
        // fetch data
    } else {
        // display error
    }
    ...
}
```

Завдання 2. Вивчіть код додатків.

HttpURLConnection

```
private String downloadUrl(String myurl) throws IOException {
    InputStream is = null;

    // Only display the first 500 characters of the retrieved web page content.
    int len = 500;

    try {
        URL url = new URL(myurl);

        HttpURLConnection conn = (HttpURLConnection) url.openConnection();

        conn.setReadTimeout(10000 /*milliseconds */);
        conn.setConnectTimeout(15000 /*milliseconds*/);

        conn.setRequestMethod("GET");
        conn.setDoInput(true);

        // Starts the query
        conn.connect();
        int response = conn.getResponseCode();
        Log.d(DEBUG_TAG, "The response is:" + response);

        is = conn.getInputStream();

        // Convert the InputStream into a string
        String contentAsString = readIt(is, len);

        return contentAsString;

        // Makes sure that the InputStream is closed after the app is finished using it.
    } finally {
        if (is! = null) {
            is.close ();
        }
    }
}
```

Перетворення отриманої інформації до типу String.

```java
public String readIt(InputStream stream, int len)
    throws IOException, UnsupportedEncodingException {

    Reader reader = null;
    reader = new InputStreamReader(stream, "UTF-8");
    char[] buffer = new char[len];
    reader.read(buffer);

    return new String(buffer);
}
```

HTTP GET запит.

```java
// Створюємо HttpClient
HttpClient client = new DefaultHttpClient();

// Створюємо об'єкт HttpGet
HttpGet request = new HttpGet( "http://www.example.com");

// Виконуємо HTTP запит
HttpResponse response;
try {
    response = client.execute(request);
    Log.d("Response of GET request", response.toString());
} catch (ClientProtocolException e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
} catch (IOException e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
}
```

Взаємодія з сервером через сокети.

```java
public class Requester extends Thread {
    Socket requestSocket;
    String message;
    StringBuilder returnStringBuffer = new StringBuilder();
    Message lmsg;
    int ch;

    @Override
    public void run() {
        try {
            this.requestSocket = new Socket("remote.servername.com", 13);
            InputStreamReader isr = new InputStreamReader(this.requestSocket.GetInputStream(),
            ↪ "ISO-8859-1");

            while ((this.ch = isr.read())! = -1) {
                this.returnStringBuffer.append((char) this.ch);
            }

            this.message = this.returnStringBuffer.toString();
            this.lmsg = new Message();
            this.lmsg.obj = this.message;
            this.lmsg.what = 0;

            h.sendMessage(this.lmsg);
```

```
24                this.requestSocket.close();
25            } catch (Exception ee) {
26                Log.d("sample application", "failed to read data" + ee.getMessage());
27            }
28        }
29    }
```

Завдання 3. Робота з зовнішніми файлами.

Розробити мобільний додаток, що дозволяє користувачеві асинхронно завантажувати файли журналу. Файли зберігаються на сервері в форматі PDF і розташовані за адресою: `http://chemengine.kpi.ua`.

Необхідно передбачити повідомлення про відсутність файлу. У разі якщо файл не знайдений, відповідь від сервера буде містити головну сторінку сайту.

Приклади посилань:

1. `http://chemengine.kpi.ua/issue/view/13836` – повернуто PDF файл.
2. `http://chemengine.kpi.ua/issue/view/13837` – файл не знайдений, повернута головна сторінка сайту.

Файли повинні зберігатися на пристрої в папці, яка створюється при першому запуску програми. Після закінчення завантаження файлу повинна ставати доступною кнопка «Дивитися» і кнопка «Видалити».

При натисканні на кнопку «Дивитися» має відбуватися відкриття збереженого на пристрої файлу. Передбачити помилку, якщо в ньому не встановлено додаток, що відкриває PDF файли.

При натисканні на кнопку «Видалити» завантажений документ повинен видалятися з пристрою.

Завдання 4. Зберігання та читання налаштувань.

При запуску програми користувачеві повинно виводитися спливаюче напівпрозоре повідомлення (`popupWindow`), з короткою інструкцією із користування послугою (можете написати випадковий текст), чекбокс «Більше не показувати» і кнопкою «ОК».

Якщо чекбокс був відзначений і натиснута кнопка ОК, необхідно зробити збереження даного параметра використовуючи `SharedPreferences`. При наступному запуску програми проводити перевірку параметра, і не виводити спливаюче повідомлення, якщо чекбокс був відзначений.

Хід роботи.

Завдання 1. У першому завданні продемонстровано базовий спосіб перевірки наявності підключення до мережі Інтернет у додатку Android. Для цього використовується системний сервіс `ConnectivityManager`, який дозволяє отримати детальну інформацію про поточний стан мережевих з'єднань пристрою. За допомогою методу `getActiveNetworkInfo()` створюється об'єкт, що містить актуальні дані про активну мережу. Далі умовна конструкція перевіряє, чи цей об'єкт взагалі існує (чи не дорівнює він `null`) та чи дійсно встановлено з'єднання за допомогою методу `isConnected()`. Якщо перевірка успішна, програма може безпечно виконувати завантаження даних із сервера, а в іншому випадку розробник має передбачити виведення повідомлення про відсутність Інтернету, щоб уникнути аварійного завершення додатку.

Завдання 2. Перший фрагмент коду другого завдання ілюструє процес завантаження даних за вказаною URL-адресою з використанням стандартного класу `HttpURLConnection`. Спочатку створюється об'єкт URL, який відкриває з'єднання. Для забезпечення стабільності роботи програми налаштовуються таймаути: 10 секунд на читання та 15 секунд на підключення. Також вказується метод запиту `GET`, який призначений саме для отримання інформації. Після виклику методу `connect()` програма ініціює зв'язок і отримує код відповіді від сервера для підтвердження успішності операції. Вхідний байтовий потік даних (`InputStream`) передається у допоміжний метод для перетворення тексту, після чого гарантовано закривається у блоці `finally`, що є обов'язковою практикою для запобігання витоку оперативної пам'яті.

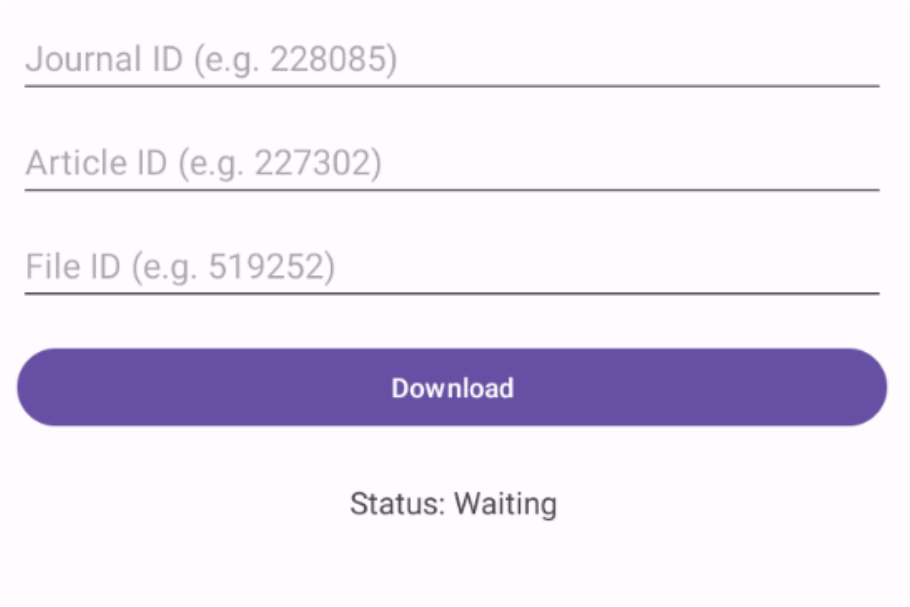
Наступний фрагмент коду демонструє допоміжний метод `readIt`, який бере на себе завдання конвертації сирого байтового потоку даних (`InputStream`) у зручний текстовий формат `String`. Для цього використовується об'єкт `InputStreamReader`, який зчитує байти та коректно декодує їх у символи згідно з кодуванням UTF-8. Зчитування відбувається у заздалегідь створений символьний масив (буфер) фіксованої довжини, визначеної вхідним параметром. Після завершення читання цей масив перетворюється на звичайний рядок і повертається як кінцевий результат роботи функції.

Третій приклад показує альтернативний спосіб виконання HTTP-запитів за допомогою об'єктів `HttpClient` та `HttpGet`. Хоча цей підхід з використанням бібліотеки Apache HTTP вважається застарілим і вилучений із сучасних версій Android SDK, він добре демонструє логіку формування мережевого запиту. Ство-

рюється клієнт та об'єкт запиту із вказаною адресою. Безпосереднє виконання запиту відбувається через виклик методу `execute()`, який повертає серверу відповідь у вигляді об'єкта `HttpResponse`. Увесь цей процес обов'язково обгорнуто у блок `try-catch` для перехоплення специфічних помилок протоколу або загальних проблем із введенням-виведенням під час передачі даних по мережі.

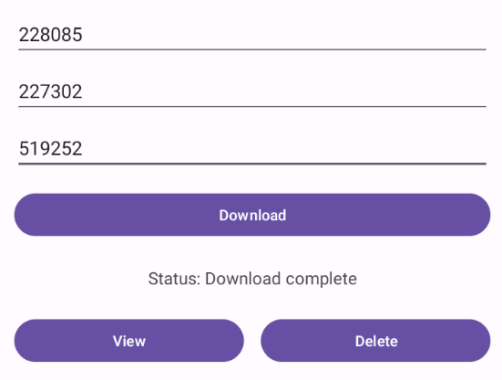
Останній фрагмент коду описує низькорівневу взаємодію із сервером за допомогою мережевих сокетів (Socket). Ця операція виконується в окремому фоновому потоці, оскільки клас `Requester` успадковується від базового класу `Thread`. Програма встановлює пряме з'єднання з віддаленим сервером за жорстко заданою адресою та портом (у цьому випадку порт 13). За допомогою циклу `while` дані посимвольно зчитуються з потоку та додаються до об'єкта `StringBuilder`, поступово формуючи цілісне повідомлення. Отриманий текст пакується у спеціальний об'єкт `Message` і відправляється до головного потоку програми через систему повідомлень (`Handler`) для подальшого відображення в інтерфейсі, після чого сокет безпечно закривається.

Завдання 3. Для виконання третього завдання було розроблено механізм асинхронного завантаження файлів з урахуванням сучасної структури цільового веб-сайту. Замість одного загального ідентифікатора у графічному інтерфейсі було реалізовано три окремі текстові поля для введення ідентифікаторів випуску журналу, конкретної статті та безпосередньо самого файлу.

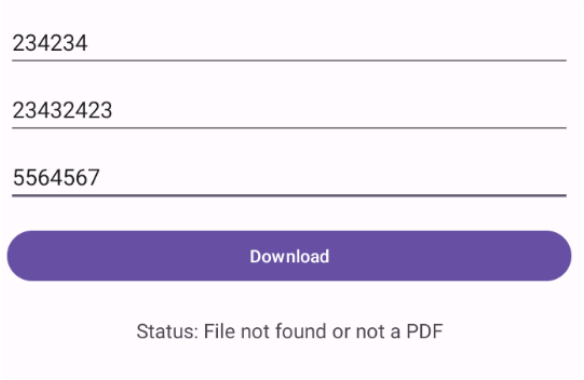


Після натискання кнопки завантаження програма зчитує ці дані та конструює точне пряме посилання на завантаження PDF-документа. Сам процес мережевого запиту виконується у фоновому потоці за допомогою `Kotlin Coroutines` (`Dispatchers.IO`), що гарантує відсутність зависань головного потоку інтерфей-

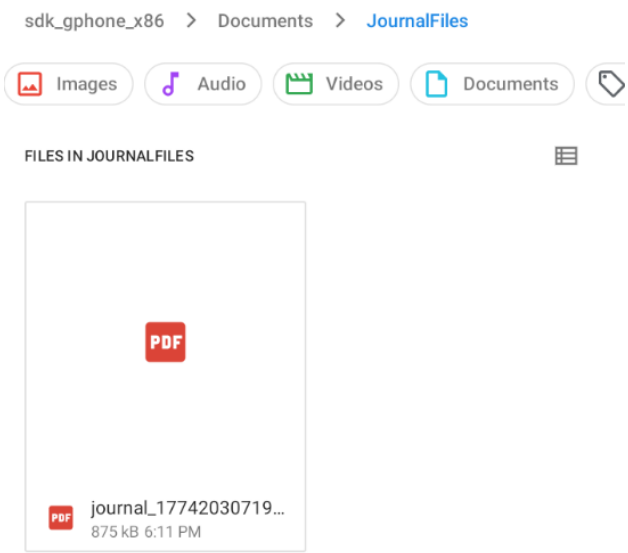
су. Для перевірки наявності файлу аналізується заголовок HTTP-відповіді сервера `Content-Type`. Якщо сервер повертає `application/pdf`, байтовий потік зчитується та зберігається на пристрій.



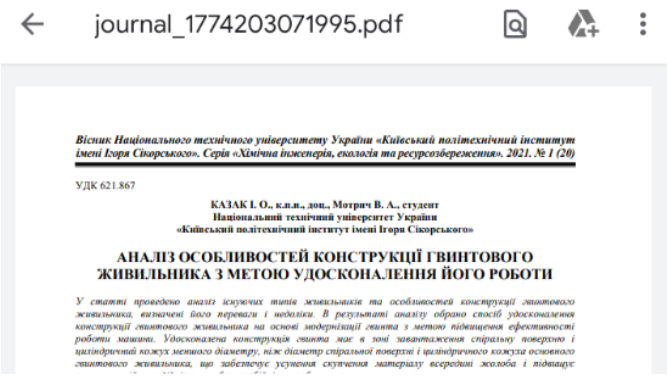
Якщо ж ні – програма розуміє, що файл відсутній (наприклад, повертається HTML-сторінка), і повідомляє про це користувача.



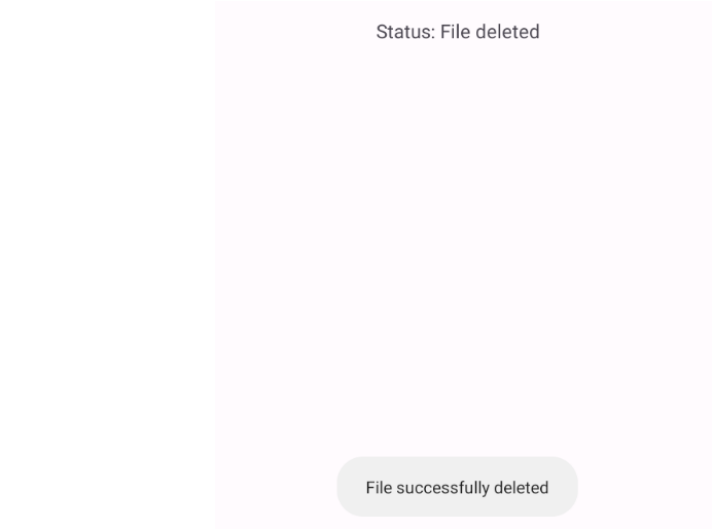
Для зручності файли зберігаються у спеціально створеній загальнодоступній директорії `Documents/JournalFiles`, шлях до якої отримується через системний клас `Environment`.



Після успішного завантаження на екрані з'являються кнопки керування. Для безпечного відкриття збереженого файлу сторонніми програмами налаштовано системний компонент `FileProvider` із зазначенням дозволених шляхів до загальнодоступної пам'яті. У разі відсутності на пристрої програми для читання PDF перехоплюється виняток `ActivityNotFoundException` із виведенням відповідного `Toast`-сповіщення.

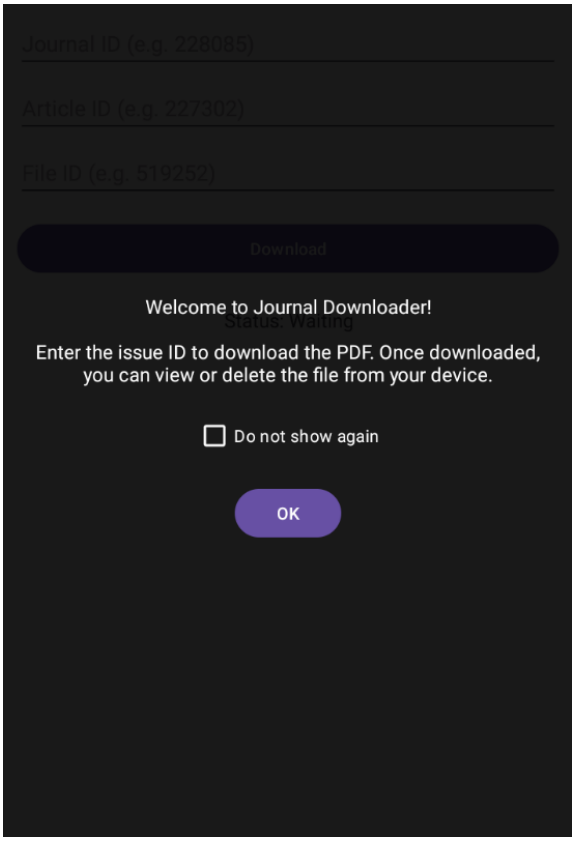


Функція видалення надійно стирає фізичний файл зі сховища та ховає кнопки керування до наступного завантаження.



Завдання 4. Четверте завдання присвячене реалізації збереження локальних налаштувань додатку за допомогою системного інтерфейсу `SharedPreferences`. Для цього було створено окрему XML-розмітку напівпрозорого спливаючого вікна (`PopupWindow`), яке містить вітальну інформацію, чекбокс для відмови від подальших показів та кнопку підтвердження. Під час ініціалізації головної активності програма звертається до файла налаштувань та перевіряє значення спеціального булевого прапорця. Якщо воно істинне (що є значенням за замовчуванням при першому встановленні додатку), вікно програмно прив'язується до кореневого елемента інтерфейсу та виводиться поверх нього. Коли користувач натискає

кнопку згоди, попередньо відмітивши чекбокс «Більше не показувати», додаток записує нове хибне значення цього прапорця у `SharedPreferences`. Таким чином, при всіх наступних запусках програми перевірка не проходитиме, і спливаюче повідомлення більше не перекриватиме екран, що повністю виконує поставлені вимоги щодо читання та збереження налаштувань.



Висновок. Під час виконання лабораторної роботи було практично закріплено навички роботи з файловою системою пристрою, мережевими запитами та механізмами збереження налаштувань в операційній системі Android. Було розроблено додаток, який асинхронно завантажує PDF-документи з веб-сервера, використовуючи сучасний підхід з `Kotlin Coroutines`, та зберігає їх у публічну директорію пристрою. Окрему увагу було приділено налаштуванню системного компонента `FileProvider` для безпечної передачі завантажених файлів стороннім програмам через неявні наміри (`Intents`). Крім того, було успішно реалізовано механізм збереження стану користувацького інтерфейсу за допомогою `SharedPreferences`, що дозволило керувати відображенням вітального спливаючого вікна (`PopupWindow`) залежно від попередніх дій користувача. Загалом, отриманий досвід дозволяє створювати більш комплексні мобільні додатки, здатні ефективно взаємодіяти із зовнішнім середовищем та надійно зберігати свій стан між сеансами роботи.

Контрольні запитання.

1. **Для чого використовуються об'єкти класу Intent?**
   Об'єкти класу `Intent` (наміри) використовуються як механізм обміну повідомленнями між різними компонентами системи Android. Вони дозволяють запускати нові активності, служби або відправляти широкомовні повідомлення (`Broadcasts`), а також передавати дані між цими компонентами під час їхнього виклику.
2. **Як виконати явний виклик Activity?**
   Явний виклик Activity виконується шляхом створення об'єкта `Intent`, якому чітко передається контекст поточного компонента та клас цільової активності (наприклад, `Intent(this, SecondActivity::class.java)`). Після цього викликається метод `startActivity()`, куди передається створений намір.
3. **Як виконати неявний виклик Activity?**
   Неявний виклик Activity здійснюється без вказівки конкретного цільового класу. Замість цього в об'єкті `Intent` задається певна дія (Action, наприклад `Intent.ACTION_VIEW`), категорія та тип даних (наприклад, MIME-тип файлу PDF). Операційна система сама аналізує запит, підбирає та пропонує користувачеві додатки, які зареєстровані як такі, що здатні обробити цю конкретну дію.
4. **Які стани передбачені життєвим циклом Activity?**
   Життєвий цикл Activity включає кілька основних станів: створена (Created), запущена і видима на екрані (Started), активна і у фокусі користувача (Resumed), призупинена через часткове перекриття іншим вікном (Paused), повністю зупинена і невидима (Stopped) та остаточно знищена системою (Destroyed).
5. **Які методи автоматично спрацьовують при зміні стану Activity? Як можна використовувати ці методи?**
   При зміні станів автоматично викликаються методи зворотного виклику: `onCreate` (для початкової ініціалізації інтерфейсу), `onStart` (коли активність стає видимою), `onResume` (початок взаємодії з користувачем), `onPause` (втрата фокусу), `onStop` (активність більше не видима), `onDestroy` (перед повним знищенням) та `onRestart`. Розробники перевизначають ці методи для виділення або звільнення ресурсів, збереження проміжних даних користувача, зупинки чи запуску фонових потоків (наприклад, програвання аудіо) залежно від того, чи користується людина додатком у даний момент.

6. **Як виконується передача даних за допомогою Intent?**

   Передача даних здійснюється за допомогою методу `putExtra()`, який викликається для об'єкта `Intent` перед його запуском. Цей метод приймає пару «ключ-значення», де ключ є унікальним текстовим ідентифікатором, а значення може бути базовим типом даних (число, рядок) або складним об'єктом, що реалізує інтерфейс Serializable чи Parcelable. У цільовій активності ці дані витягуються через властивість `intent` за допомогою відповідних методів, таких як `getStringExtra` або `getIntExtra`.

7. **В якому вигляді зберігаються даних за допомогою класу SharedPreferences?**

   Дані в `SharedPreferences` зберігаються у вигляді пар «ключ-значення» (key-value) у спеціальному XML-файлі, який знаходиться у внутрішній закритій пам'яті додатка. Цей механізм підтримує збереження лише базових (примітивних) типів даних: логічних значень (boolean), чисел (int, float, long), текстових рядків (String) та наборів рядків (Set<String>).

8. **В яких випадках доцільно зберігати дані за допомогою класу SharedPreferences?**

   Використання `SharedPreferences` є доцільним для збереження невеликих обсягів простих даних, які повинні зберігатися між запусками програми. Це ідеальний інструмент для збереження користувацьких налаштувань (наприклад, теми оформлення чи мови інтерфейсу), станів елементів керування (чи показувати вітальне спливаюче вікно, стан прапорців) або простих маркерів авторизації, коли створення і підтримка повноцінної бази даних SQLite є невиправдано складною та надлишковою задачею.

Лістинг.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />

<application
android:allowBackup="true"
android:icon="@mipmap/ic_launcher"
android:label="@string/app_name"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportsRtl="true"
android:theme="@style/Theme.KPIMMOS"
android:usesCleartextTraffic="true">
<activity
android:name=".MainActivity"
android:exported="true">
<intent-filter>
<action android:name="android.intent.action.MAIN" />

<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>

<provider
android:name="androidx.core.content.FileProvider"
android:authorities="${applicationId}.provider"
android:exported="false"
android:grantUriPermissions="true">
<meta-data
android:name="android.support.FILE_PROVIDER_PATHS"
android:resource="@xml/file_paths" />
</provider>
</application>

</manifest>
```

MainActivity.kt

```
package ua.kpi.lab5

import android.content.ActivityNotFoundException
import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.os.Environment
import android.view.Gravity
import android.view.LayoutInflater
import android.view.View
import android.widget.Button
import android.widget.CheckBox
import android.widget.EditText
import android.widget.PopupWindow
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.FileProvider
```

```
19  import androidx.lifecycle.lifecycleScope
20  import kotlinx.coroutines.Dispatchers
21  import kotlinx.coroutines.launch
22  import kotlinx.coroutines.withContext
23  import java.io.File
24  import java.io.FileOutputStream
25  import java.net.HttpURLConnection
26  import java.net.URL
27
28  class MainActivity : AppCompatActivity() {
29
30      private lateinit var etJournalId: EditText
31      private lateinit var etArticleId: EditText
32      private lateinit var etFileId: EditText
33      private lateinit var tvStatus: TextView
34      private lateinit var btnDownload: Button
35      private lateinit var btnView: Button
36      private lateinit var btnDelete: Button
37
38      private var currentDownloadedFile: File? = null
39      private val folderName = "JournalFiles"
40
41      override fun onCreate(savedInstanceState: Bundle?) {
42          super.onCreate(savedInstanceState)
43          setContentView(R.layout.activity_main)
44
45          etJournalId = findViewById(R.id.etJournalId)
46          etArticleId = findViewById(R.id.etArticleId)
47          etFileId = findViewById(R.id.etFileId)
48          tvStatus = findViewById(R.id.tvStatus)
49          btnDownload = findViewById(R.id.btnDownload)
50          btnView = findViewById(R.id.btnView)
51          btnDelete = findViewById(R.id.btnDelete)
52
53          val storageDir = File(
54          Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_DOCUMENTS),
55          folderName
56          )
57          if (!storageDir.exists()) {
58              storageDir.mkdirs()
59          }
60
61          val sharedPrefs = getSharedPreferences("AppPreferences", Context.MODE_PRIVATE)
62          val showTutorial = sharedPrefs.getBoolean("show_tutorial", true)
63          if (showTutorial) {
64              findViewById<View>(android.R.id.content).post {
65                  showTutorialPopup()
66              }
67          }
68
69          btnDownload.setOnClickListener {
70              val journalId = etJournalId.text.toString().trim()
71              val articleId = etArticleId.text.toString().trim()
72              val fileId = etFileId.text.toString().trim()
73
74              if (journalId.isNotEmpty() && articleId.isNotEmpty() && fileId.isNotEmpty()) {
75                  val fileUrl =
76                  "https://chemengine.kpi.ua/article/download/$journalId/$articleId/$fileId"
77                  downloadPdfFile(fileUrl, storageDir)
78              } else {
79                  Toast.makeText(this, "Please enter all three IDs", Toast.LENGTH_SHORT).show()
80              }
81          }
```

```
        btnView.setOnClickListener {
            currentDownloadedFile?.let { file ->
                openPdfFile(file)
            }
        }

        btnDelete.setOnClickListener {
            currentDownloadedFile?.let { file ->
                if (file.exists() && file.delete()) {
                    tvStatus.text = "Status: File deleted"
                    btnView.visibility = View.GONE
                    btnDelete.visibility = View.GONE
                    currentDownloadedFile = null
                    Toast.makeText(this, "File successfully deleted", Toast.LENGTH_SHORT).show()
                }
            }
        }
    }

    private fun showTutorialPopup() {
        val inflater = getSystemService(LAYOUT_INFLATER_SERVICE) as LayoutInflater
        val popupView = inflater.inflate(R.layout.popup_tutorial, null)

        val popupWindow = PopupWindow(
        popupView,
        androidx.constraintlayout.widget.ConstraintLayout.LayoutParams.MATCH_PARENT,
        androidx.constraintlayout.widget.ConstraintLayout.LayoutParams.MATCH_PARENT,
        true
        )

        val cbDoNotShowAgain = popupView.findViewById<CheckBox>(R.id.cbDoNotShowAgain)
        val btnOk = popupView.findViewById<Button>(R.id.btnOkPopup)

        btnOk.setOnClickListener {
            if (cbDoNotShowAgain.isChecked) {
                val sharedPrefs = getSharedPreferences("AppPreferences", Context.MODE_PRIVATE)
                sharedPrefs.edit().putBoolean("show_tutorial", false).apply()
            }
            popupWindow.dismiss()
        }

        val mainLayout = findViewById<View>(android.R.id.content)
        popupWindow.showAtLocation(mainLayout, Gravity.CENTER, 0, 0)
    }

    private fun downloadPdfFile(fileUrl: String, storageDir: File) {
        tvStatus.text = "Status: Downloading..."
        btnView.visibility = View.GONE
        btnDelete.visibility = View.GONE

        lifecycleScope.launch(Dispatchers.IO) {
            val fileName = "journal_${System.currentTimeMillis()}.pdf"
            val targetFile = File(storageDir, fileName)

            try {
                val url = URL(fileUrl)
                val connection = url.openConnection() as HttpURLConnection
                connection.requestMethod = "GET"
                connection.connect()

                val contentType = connection.contentType
                if (contentType != null && contentType.contains("application/pdf")) {
```

```

                    val inputStream = connection.inputStream
                    val outputStream = FileOutputStream(targetFile)

                    val buffer = ByteArray(4096)
                    var bytesRead: Int

                    while (inputStream.read(buffer).also { bytesRead = it } != -1) {
                        outputStream.write(buffer, 0, bytesRead)
                    }

                    outputStream.close()
                    inputStream.close()

                    withContext(Dispatchers.Main) {
                        tvStatus.text = "Status: Download complete"
                        currentDownloadedFile = targetFile
                        btnView.visibility = View.VISIBLE
                        btnDelete.visibility = View.VISIBLE
                    }
                } else {
                    withContext(Dispatchers.Main) {
                        tvStatus.text = "Status: File not found or not a PDF"
                    }
                }
                connection.disconnect()
            } catch (e: Exception) {
                e.printStackTrace()
                withContext(Dispatchers.Main) {
                    tvStatus.text = "Status: Download failed (${e.message})"
                }
            }
        }
    }

    private fun openPdfFile(file: File) {
        try {
            val uri = FileProvider.getUriForFile(this@MainActivity, "${packageName}.provider",
            ↪  file)
            val intent = Intent(Intent.ACTION_VIEW)
            intent.setDataAndType(uri, "application/pdf")
            intent.flags = Intent.FLAG_GRANT_READ_URI_PERMISSION
            startActivity(intent)
        } catch (e: ActivityNotFoundException) {
            Toast.makeText(
            this@MainActivity,
            "No application found to open PDF files",
            Toast.LENGTH_LONG
            ).show()
        } catch (e: Exception) {
            Toast.makeText(this@MainActivity, "Error opening file", Toast.LENGTH_SHORT).show()
        }
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
↪  xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
```

```
5   android:id="@+id/main"
6   android:layout_width="match_parent"
7   android:layout_height="match_parent"
8   android:padding="16dp"
9   tools:context=".MainActivity">
10
11  <EditText
12  android:id="@+id/etJournalId"
13  android:layout_width="match_parent"
14  android:layout_height="wrap_content"
15  android:hint="Journal ID (e.g. 228085)"
16  android:inputType="number"
17  app:layout_constraintTop_toTopOf="parent" />
18
19  <EditText
20  android:id="@+id/etArticleId"
21  android:layout_width="match_parent"
22  android:layout_height="wrap_content"
23  android:layout_marginTop="8dp"
24  android:hint="Article ID (e.g. 227302)"
25  android:inputType="number"
26  app:layout_constraintTop_toBottomOf="@id/etJournalId" />
27
28  <EditText
29  android:id="@+id/etFileId"
30  android:layout_width="match_parent"
31  android:layout_height="wrap_content"
32  android:layout_marginTop="8dp"
33  android:hint="File ID (e.g. 519252)"
34  android:inputType="number"
35  app:layout_constraintTop_toBottomOf="@id/etArticleId" />
36
37  <Button
38  android:id="@+id/btnDownload"
39  android:layout_width="match_parent"
40  android:layout_height="wrap_content"
41  android:layout_marginTop="16dp"
42  android:text="Download"
43  app:layout_constraintTop_toBottomOf="@id/etFileId" />
44
45  <TextView
46  android:id="@+id/tvStatus"
47  android:layout_width="match_parent"
48  android:layout_height="wrap_content"
49  android:layout_marginTop="24dp"
50  android:text="Status: Waiting"
51  android:textAlignment="center"
52  android:textSize="16sp"
53  app:layout_constraintTop_toBottomOf="@id/btnDownload" />
54
55  <Button
56  android:id="@+id/btnView"
57  android:layout_width="0dp"
58  android:layout_height="wrap_content"
59  android:layout_marginTop="24dp"
60  android:layout_marginEnd="8dp"
61  android:text="View"
62  android:visibility="gone"
63  app:layout_constraintEnd_toStartOf="@id/btnDelete"
64  app:layout_constraintStart_toStartOf="parent"
65  app:layout_constraintTop_toBottomOf="@id/tvStatus" />
66
67  <Button
```

```
android:id="@+id/btnDelete"
android:layout_width="0dp"
android:layout_height="wrap_content"
android:layout_marginStart="8dp"
android:layout_marginTop="24dp"
android:text="Delete"
android:visibility="gone"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintStart_toEndOf="@id/btnView"
app:layout_constraintTop_toBottomOf="@id/tvStatus" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

popup_tutorial.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="300dp"
android:layout_height="wrap_content"
android:background="#E6000000"
android:gravity="center"
android:orientation="vertical"
android:padding="24dp">

<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginBottom="16dp"
android:text="Welcome to Journal Downloader!\n\nEnter the issue ID to download the PDF. Once
↪ downloaded, you can view or delete the file from your device."
android:textAlignment="center"
android:textColor="#FFFFFF"
android:textSize="16sp" />

<CheckBox
android:id="@+id/cbDoNotShowAgain"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginBottom="16dp"
android:buttonTint="#FFFFFF"
android:text="Do not show again"
android:textColor="#FFFFFF" />

<Button
android:id="@+id/btnOkPopup"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="OK" />

</LinearLayout>
```

file_paths.xml

```
<?xml version="1.0" encoding="utf-8"?>
<paths>
<external-path
name="public_documents"
path="Documents/JournalFiles" />
</paths>
```